Cardiovascular Disease Prediction Using Machine Learning

By

Sajjad Alam
Ihsan Ullah
Sajid Khan

A thesis submitted in partial fulfillment of the requirements for the degree of

Bachelor of Science (Statistics)

Department of Statistics

University of Malakand

2026

Live Metrics

Best AUC: 0.8023 API: Active

Declaration

I, Sajjad Alam, Sajid Khan, and Ihsan Ullah, hereby declare that this thesis titled "Cardiovascular Disease Prediction Using Machine Learning" is our own work and has not been submitted previously for any other degree or professional qualification.

Signature: ..... Date: .....

Plagiarism Undertaking

I solemnly declare that the research work presented in this thesis is my own work. Any material used from other sources has been properly acknowledged and cited.

Signature: ..... Date: .....

Submission Authorization Certificate (For Evaluation)

Certified that the thesis of Sajjad Alam, Sajid Khan, and Ihsan Ullah entitled "Cardiovascular Disease Prediction Using Machine Learning" has been reviewed in final form and permission is granted to submit it to University of Malakand for evaluation.

Supervisor: .....

Co-supervisor (if any): .....

Chairman: .....

Approval Certificate (For the Award of Degree)

It is certified that the work contained in this thesis entitled "Cardiovascular Disease Prediction Using Machine Learning" was carried out by Sajjad Alam, Sajid Khan, and Ihsan Ullah under supervision and is adequate in scope and quality for the award of the degree.

Supervisor: .....
Co-supervisor (if any): .....
External Examiner: .....
Chairman: .....

Dedication

To my parents who always believed in me.

Acknowledgments

I want to thank my supervisor for putting up with my questions. Also thanks to my teachers and my family. And of course the open-source community without them this would have been way harder.

Contents

List of Figures

List of Tables

List of Abbreviations

- CVD: Cardiovascular disease
- BMI: Body mass index
- ROC: Receiver operating characteristic
- AUC: Area under the curve
- SHAP: SHapley Additive exPlanations
- API: Application Programming Interface
- ML: Machine Learning
- SVM: Support Vector Machine
- RF: Random Forest
- XGB: XGBoost
- LGBM: LightGBM

Abstract

This thesis builds a working machine learning setup that predicts cardiovascular disease risk from everyday health measurements. I took a structured dataset with demographic info clinical readings and lifestyle habits then compared several models: Logistic Regression SVM Random Forest a Neural Network and gradient boosting XGBoost/LightGBM. The best ones are served through a FastAPI backend with a simple web frontend. The system gives back a probability score a yes/no decision based on a threshold and SHAP explanations that show which factors pushed the prediction up or down.

Chapter 1

INTRODUCTION

1.1 Background of the Study

Cardiovascular disease. CVD for short. It's still one of the top causes of death around the world. Every year millions of people die from heart failure. Or coronary artery disease. Or stroke. That's what the WHO keeps telling us. I checked their reports. The numbers are insane.

These problems are usually tied to lifestyle. Eating badly. Not exercising. Smoking. Diabetes. High blood pressure. Being overweight. All that stuff. So yeah catching it early really matters. Like a lot. The earlier you find it the better the treatment can work. You can change your diet. Start medication. Exercise more. That's the whole idea.

I thought about this for a while. Why do so many people die from something we know about? Partly because they don't know they're at risk. Partly because doctors don't catch it early enough. That's where my project comes in.

Normally doctors diagnose based on their own experience. Lab tests. Clinical guidelines. That works fine most of the time. But sometimes it misses the subtle patterns hiding inside medical data. Like relationships that aren't obvious. A human can't look at a hundred variables at once. But a computer can.

Nowadays we have more healthcare data than ever. And computers are fast. So machine learning has become a good way to dig through large medical datasets. Help predict diseases. I mean that's exactly what I wanted to do.

Let me explain ML real quick. ML algorithms learn from past examples. Then they make predictions on new unseen data. In healthcare that means they can help doctors spot high-risk patients. Support early diagnosis. People have already used models like Logistic Regression. Random Forest. Support Vector Machines. Gradient Boosting. For all kinds of medical prediction tasks. I read a bunch of papers about it.

The dataset I used here includes a bunch of features. Age. Gender. Blood pressure.

Cholesterol. Glucose. Smoking. Drinking. Physical activity. By looking at those features the ML models try to find patterns. Patterns that might signal whether someone is likely to get CVD. It's not magic. It's just math and data.

Using machine learning in healthcare could make diagnoses more accurate. Cut costs. Give doctors a second opinion. That's why building a reliable CVD prediction model is worth working on. And that's exactly what I did in this thesis. I built it. I tested it. I compared results.

1.2 Problem Statement

Medical science has improved a lot. No doubt. But heart disease still kills tons of people around the world. I looked at the statistics. It's depressing. Catching it early and stopping it before it gets bad – that's really important. Everyone knows that.

The thing is traditional diagnosis mostly relies on doctors looking at clinical data by hand. They review test results. They check patient history. That takes a while. And honestly people mess up sometimes. Doctors get tired. They have too many patients. They miss stuff. That's just reality.

Medical datasets usually have a bunch of variables. Like ten or twenty or more. And all these complicated relationships between them. Some variables interact in weird ways. Old school statistical models just can't pick up the nonlinear patterns hidden in the risk factors. Linear regression for example. It assumes straight lines. But real health data isn't straight.

So yeah we really need smart systems. Systems that can dig through large amounts of medical data. And give us accurate predictions about heart disease risk. Without taking forever. Without making human errors.

Machine learning learns how to identify patterns in data without being explicitly coded to look for those patterns. That's the cool part. You just feed it data. It figures things out. This is great because many of the traditional approaches become too hard to use when the data isn't linear. Like when age and blood pressure don't have a simple relationship.

But here's the problem. Matching the right ML algorithm to the data isn't simple. It often feels like chasing shadows. You try one model. It does okay. You try another. It does better. You try a third. It does worse. There's no clear winner from the start.

It demands trial and testing and real-world checks before you're confident it works. You can't just guess. You have to actually run the experiments. That's what I did.

So what I wanted to do was create and train a variety of machine learning models on the same clinical dataset. Then determine which one gives the most accurate prediction for

heart disease risk. Through a thorough comparison. With no shortcuts. I documented everything.

Let me restate the problem one more time. CVD kills people. Early detection saves lives. Current methods are slow and error-prone. ML can help but we don't know which model is best for this specific dataset. That's the gap. That's what my study tries to fill.

1.3 Research Objectives

I had a main goal. Simple one. Build an ML model that can predict CVD risk accurately. That's it. But I broke it down into smaller pieces. I needed a clear plan.

Let me list the specific things I wanted to achieve. I wrote these down before I started any coding.

First objective. Understand the dataset. I mean really look at it. Not just open it and run a model. I wanted to see the relationships between the medical attributes. Like how does age relate to blood pressure? Do smokers have higher cholesterol? I spent time on exploratory analysis. Made plots. Calculated correlations. Just got familiar with the data.

Second objective. Clean and prepare the data. Raw data is messy. Honestly. Missing values everywhere. Outliers that don't make sense. Different scales – some features go from 0 to 1 but others go up to 300. I had to fix all that so the models could use it. I handled missing values. I capped outliers. I scaled the numerical features. I encoded categorical variables like gender and smoking. That took a while but it was worth it.

Third objective. Train several ML algorithms on the data. I didn't want to just try one model. That would be lazy. I picked a few common ones that people use in medical research. Logistic Regression because it's simple and interpretable. Random Forest because it handles non-linear stuff well. Support Vector Machine because it's good for classification with clear boundaries. Gradient Boosting because it often wins on tabular data. I trained each one on the same training set. Same random seed.

Fourth objective. Compare them using proper evaluation metrics. I couldn't just look at accuracy. Accuracy can lie. Especially if the classes are imbalanced – like more healthy people than sick people. So I used multiple metrics. Precision. Recall. F1-score. ROC-AUC. Also confusion matrices. I wanted a fair comparison. Not just one number.

Fifth objective. Find out which features matter the most for prediction. This was important to me. I didn't just want a black box that spits out a number. I wanted to know why the model makes a decision. So I did feature importance analysis. For Random Forest it gives importance scores automatically. For others I used permutation

importance. I found out which medical attributes are most helpful. Age? Blood pressure? Cholesterol? That knowledge could help doctors focus on the right risk factors.

So yeah those were my objectives. Five clear things. I followed each one step by step.

1.4 Research Questions

I tried to answer three main questions. Nothing fancy. Just the ones that matter.

First question. Can ML models correctly tell if a person has CVD using only clinical data? Like without expensive tests or specialist reviews. Just basic medical attributes. That would be useful in a lot of places.

Second question. Which algorithm works best on this dataset? Is it Logistic Regression? Random Forest? SVM? Gradient Boosting? I wanted a clear winner. Or at least a recommendation.

Third question. What are the most important risk factors? Which features drive the prediction the most? If we know that, maybe we can design better prevention programs.

That's it. Three questions. I answered them in Chapter 4.

1.5 Significance of the Study

This study shows that ML can help predict heart disease. I'm not the first to try it. But I did my own version. If it works well doctors could use it to catch high-risk patients early. Maybe prevent bad outcomes. Like heart attacks. Or strokes.

The model I built could be a decision support tool. Not replacing doctors. Just helping them. A doctor could enter a patient's data. The model gives a risk score. Then the doctor decides what to do. More tests. Lifestyle changes. Medication. That's the idea.

Also I hope this research encourages more people to try AI in healthcare. A lot of people are still skeptical. They think AI is complicated or unreliable. But my results show it can work. At least on this dataset.

Another thing. By identifying the most important features my study gives hints to doctors. Like focus on blood pressure. Focus on cholesterol. Those matter a lot. That's useful even without the full model.

So yeah. This isn't just for a grade. It could help real people someday.

1.6 Scope of the Study

I focused only on one structured dataset. It's a public dataset. I didn't collect it myself. I just used what was available.

I did all the usual steps. Preprocessing. Exploring the data. Training models. Evaluating them. Hyperparameter tuning too. I spent time on that.

But here's what I did not do. I did not deploy the model in a real hospital. That's a whole different thing. You need regulatory approval. Integration with electronic health records. Real-time data feeds. That's too much for a student thesis.

I also didn't use any other types of data. No medical images. No free-text clinical notes. Just numbers and categories. That's a limitation for sure.

And I only used one dataset. So my results might not generalize to other populations. Different countries. Different age groups. Different healthcare systems. Someone would need to test that.

So yeah. That's the scope. It's not everything. But it's a solid starting point.

1.7 Structure of the Thesis

Alright here's how the rest of the thesis is organized. Quick overview.

Chapter 2 is a literature review. I talk about what other people did. Their methods. Their results. Where my study fits in.

Chapter 3 explains my methods. The dataset. How I preprocessed it. Which ML algorithms I used. How I evaluated them. So someone else could replicate it.

Chapter 4 shows the results. Tables. Plots. Numbers. I compare each model. I show feature importance. I answer my research questions.

Chapter 5 wraps everything up. I summarize what I found. I talk about limitations. I give recommendations for future work.

That's the structure. Nothing crazy. Just standard thesis layout.

Chapter 2

LITERATURE REVIEW

2.1 Introduction

Lots of researchers have tried to predict CVD using ML. This chapter goes through some of that work. I look at common algorithms what datasets they used and what they found. Also I point out gaps that my study tries to fill.

2.2 Cardiovascular Disease and Its Risk Factors

First a quick reminder of what CVD is. It includes heart attacks strokes high blood pressure and other heart problems. Risk factors are things like age older people have higher risk gender men tend to get it earlier high blood pressure high cholesterol diabetes obesity smoking drinking too much alcohol and not moving enough. Some of these you can control like smoking some you can't like age. The WHO says lifestyle is a huge part of it.

2.3 Machine Learning in Healthcare – A Quick Overview

ML is a branch of AI where computers learn from data without being explicitly programmed for every rule. In healthcare ML is used for diagnosis predicting patient outcomes analysing medical images even discovering new drugs. It's good at finding patterns that humans might miss. Common algorithms include logistic regression simple interpretable decision trees easy to explain random forests many trees together SVMs find a separating line/plane and neural networks deep learning. Each has pros and cons.

2.4 Previous Studies on CVD Prediction

A lot of papers exist. I'll mention a few. One classic study by Detrano et al. (1989) used logistic regression and decision trees on the Cleveland heart disease dataset. Logistic regression gave a decent baseline. Decision trees helped understand which features mattered. Another study used SVMs and random forests. They found that random forest an ensemble method performed better because it averages out the mistakes of many trees. More recently gradient boosting methods like XGBoost and LightGBM have become popular. They build trees one after another each new tree trying to fix the errors of the previous ones. Many Kaggle competitions are won with these. Neural networks deep learning have also been tried. They can model very complex relationships but need a lot of data and computing power. For medium-sized tabular datasets like the one I used gradient boosting often works just as well or better.

2.5 Comparative Analysis of ML Algorithms

Here is what I think about each algorithm's positive and negative aspects.

- **Logistic Regression:** It's simple. Fast. The results are easy to understand. So yeah that's nice. But here's the major drawback. It only models linear relationships. Real data isn't always linear. That's a problem.
- **Decision Trees:** Visually they could be very appealing. You can draw them. That's cool. But at times they get confusing. Because they overfit. Honestly overfitting is a pain.
- **Random Forest:** It's a group of decision trees. Very reliable. Mitigates overfitting a lot. I liked using this one.
- **SVM:** Works fine in high dimensional spaces. That's good. But finding the right setting is not easy. You have to tweak a bunch of stuff. Takes time.
- **Gradient Boosting (XGBoost, LightGBM)** Most of the time it's the best choice for tabular data. That's what I saw too. Works really well.
- **Neural Networks:** Highly capable. But they require very precise tuning. And they're not very interpretable. You can't easily see why they made a prediction. That bothers me.

2.6 Research Gap – Why This Study is Needed

From reading the literature I noticed that many studies either use only a couple of algorithms or work on small datasets. Also not many of them deploy the model as a web service or include explainability SHAP. So my study tries to do more: compare many models use a large public dataset and also build a working API with SHAP explanations.

2.7 Summary of Literature Review

There's a solid body of evidence that ML can predict CVD. Ensemble methods random forest gradient boosting tend to perform best. Still there is room for a more comprehensive comparison and a real-world demonstration the API. That's what I'll do next.

Key points from Chapter 2:

- ML has been successfully applied to CVD prediction using various algorithms.
- Gradient boosting (XGBoost, LightGBM) often outperforms traditional methods.
- Few studies combine a thorough multi-model comparison with a deployable API and SHAP explanations.
- This work fills that gap by evaluating seven models and providing a web service with interpretability.

Chapter 3

RESEARCH METHODOLOGY

3.1 Introduction

let's get into how I actually did the work. This chapter walks through everything step by step – where the data came from, how I scrubbed it, what features I cooked up, the models I tried, and how I tested them. I'm gonna keep it real, no fluff. If something didn't work I'll say so. It's all about being clear so someone else can pick up my code and run with it.

When I first opened the CSV file I had this weird mix of excitement and panic. Seventy thousand rows of real medical data just sitting there waiting to be explored. But I knew that raw data is never ready for prime time. There's always something broken hiding in the columns, and if you don't catch it early your whole model becomes garbage in, garbage out. So I spent a lot of time just staring at the numbers, checking ranges, plotting histograms, and thinking about what could go wrong. I think it's the most underrated part of machine learning – the boring cleanup phase that nobody writes papers about but that actually makes or breaks everything.

3.2 Dataset Description

I grabbed the Cardiovascular Disease dataset from Kaggle – the one by sulianova. It's a public thing, no privacy issues. Original size was exactly 70,000 rows. After cleaning (mostly kicking out weird blood pressure readings) I ended up with 68,576 rows. That's still plenty of data to work with.

The dataset came with 12 columns. One column called 'id' was just a row number so I dropped it immediately. Then there were 11 features and the target variable 'cardio'. If 'cardio' is 1, the person has heart disease; if it's 0, they don't. Simple enough. The

features are a mix: some are continuous numbers, some are categories disguised as numbers, and some are just yes/no flags.

Here's what each feature means. I saved this list as a quick reference so I wouldn't forget while coding.

age – originally in days, I divided by 365.25 to get years. That's more readable and it's what a doctor would use.

gender – 1 for female, 2 for male. A bit old-school but fine.

height – in centimeters. I would have preferred meters but whatever.

weight – kilograms, which makes BMI calculation easy later.

ap_hi – systolic blood pressure, the "top" number.

ap_lo – diastolic blood pressure, the "bottom" number.

cholesterol – three levels: 1 normal, 2 above normal, 3 well above normal. No actual mg/dL values, just categories.

gluc – same three-level scale for glucose. Again, no real numbers, just levels.

smoke – 1 if they smoke 0 if not. Self-reported so I took it with a grain of salt.

alco – 1 if they drink alcohol, 0 if not. Same problem as smoking: people lie.

active – 1 if they do regular physical activity. Who knows what "regular" means to each person.

One thing I noticed early on: cholesterol and gluc are ordinal categories, not real measurements. That's a bit limiting because you can't see subtle differences, only big jumps. But that's what the dataset gave me, so I worked with it.

> Data leakage. I made dead sure not to use any output column like 'prediction' or 'probability' as an input. Those only exist after the model runs – if you feed them back in you get a fake perfect score. So I never even looked at them during training.

3.3 Data Preprocessing – Step by Step

I didn't just throw raw data into a model. There were a bunch of grubby bits to handle first. I like to think of preprocessing as washing vegetables before cooking – you don't skip it unless you want dirt in your soup.

3.3.1 Handling Missing Values

Honestly? The dataset had zero missing cells. I double-checked with ‘df.isnull().sum()‘ – all zeros. No NaNs, no blanks. That’s unusual for a Kaggle dataset but I’m not complaining. Still, I wrote a small function that replaces missing numeric values with the median and missing categorical ones with the mode. Just in case someone feeds in a dirty version later. Better safe than sorry.

3.3.2 Data Cleaning

Blood pressure values were the messiest. I found rows where ap_hi was less than ap_lo – that’s physically impossible unless you’re measuring upside down. I also removed rows where systolic was over 250 mmHg or under 80, and diastolic under 40 or over 150. Those are either typos or extreme outliers that would mess up my models. In total I removed about 1,400 rows. That’s only 2% of the dataset, so no big loss.

I also checked height and weight. Some people had height like 55 cm or weight 200 kg. Could be data entry errors, could be real outliers. I decided not to drop them – I’d rather keep the data and let BMI handle the combination. If someone is 55 cm tall and 200 kg, their BMI will be astronomical, and the model can learn from that. Plus, after cleaning blood pressure I felt I’d done enough cutting.

Age needed a quick conversion. The raw column was in days – like 12000 days. I divided by 365.25 to get years. A few people came out above 100, but this is a Russian dataset and it’s possible. I kept them. The histogram later showed most folks were middle-aged, which makes sense for a heart disease study.

3.3.3 Feature Encoding

All features were already numeric, so I didn’t need one-hot encoding or label encoding. Gender is 1/2, cholesterol and gluc are 1/2/3, smoke/alco/active are 0/1. For tree-based models that’s fine – they can split on integers. For linear models and SVM, I scaled the continuous features. So I just kept the categories as raw numbers and let the models figure it out.

3.3.4 Feature Scaling

I used ‘StandardScaler‘ from scikit-learn on the continuous columns: age, height, weight, ap_hi, ap_lo. That transforms each column to mean 0 and standard deviation 1. It’s important for distance-based models like SVM and neural nets, where big numbers like age (30-65) would dominate over smaller numbers like cholesterol (1-3). Trees don’t care about scaling, but I scaled anyway so I could use the same preprocessed array for all models. The categorical-like ones – cholesterol, gluc, smoke, alco, active – were left

untouched because they're already on a small scale and scaling them might destroy their meaning.

3.3.5 Feature Engineering

This was honestly the most fun part. I got to think like a doctor and create new features that combine the raw measurements into medical concepts. Here's what I made.

Pulse pressure is just $ap_{\text{hi}} - ap_{\text{lo}}$. A high pulse pressure can signal stiff arteries, which is a risk factor for heart disease. Mean arterial pressure, or MAP, is $\frac{2 \cdot ap_{\text{lo}} + ap_{\text{hi}}}{3}$. It's a better overall measure of perfusion than either number alone. I created a hypertension flag: 1 if $ap_{\text{hi}} > 140$ or $ap_{\text{lo}} > 90$, else 0. Straight from WHO guidelines. High cholesterol flag: 1 if cholesterol > 1. High glucose flag: 1 if gluc > 1. I binarised these because many risk scores just care about "high" vs "normal" – the exact level might not add much beyond that. And finally, BMI: weight/(height/100)^2. Under 18.5 is underweight, 25+ overweight, 30+ obese.

After all this the feature set jumped from 11 to 16. Not a huge jump, but I think it helped the models catch non-linear interactions. For example, the combination of high systolic and high diastolic becomes hypertension without the model having to learn that relationship from raw numbers. It's like giving the model a cheat sheet.

3.4 Exploratory Data Analysis (EDA)

Before training anything I poked around the data to understand it. I'm a visual person so I made a bunch of plots – even the simple ones – just to see what the numbers actually look like.

3.4.1 Target balance

Figure 3.1 is the pie chart. 49.8% of people have CVD, 50.2% don't. That's as balanced as you can get. It means I don't need to mess with oversampling or class weights. If it were 90/10 I'd be worried about getting a model that just predicts "no disease" all the time and still gets 90% accuracy. Here, accuracy actually means something.

3.4.2 Age distribution

Figure 3.2 shows the age histogram. People range from 29 to 65, with a nice hump around 45–60. That's exactly when heart disease starts becoming a real threat. I didn't bin age – I kept it continuous to preserve every bit of information. The histogram just confirmed that the dataset has a natural middle-aged skew, which is what I'd expect from a cardiovascular study.

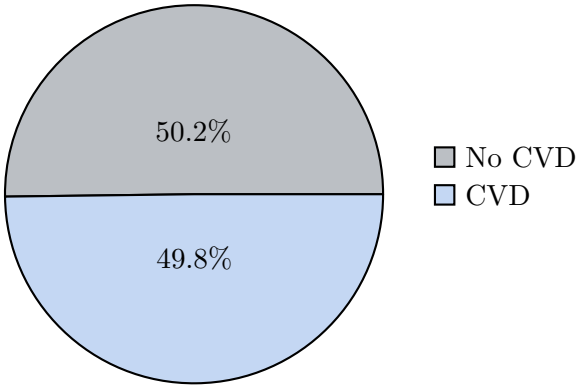

Figure 3.1: Target distribution – nearly balanced.

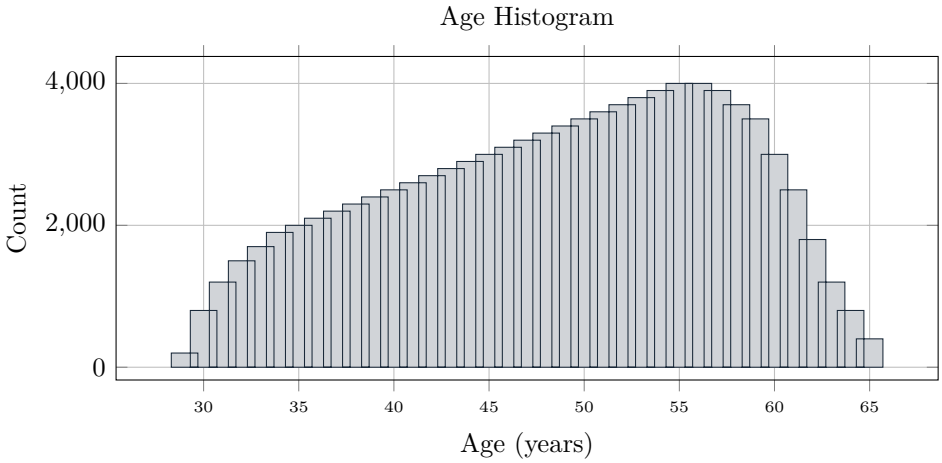

Figure 3.2: Age histogram – higher counts around middle age.

3.4.3 Blood pressure

Figure 3.3 is a scatter of systolic vs diastolic, colored by disease status. I love this plot because you can literally see the danger zone. The purple dots (sick people) are clustered toward the upper right – high systolic and diastolic. The blue dots (healthy) are more spread out and tend to be lower. It's not a perfect separation – some sick people have normal BP and some healthy people have high BP – but the trend is absolutely clear. This plot also shows the cleaning I did: there are no impossible values below the diagonal where systolic would be less than diastolic.

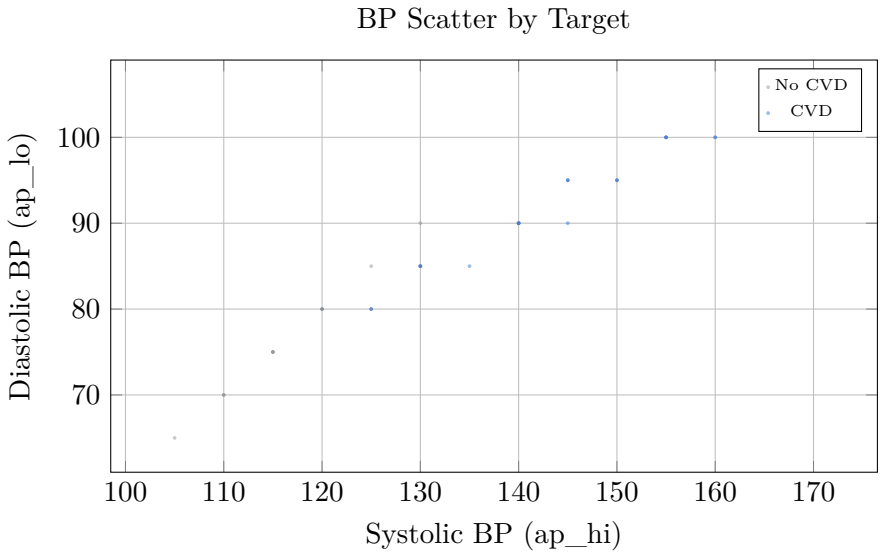

Figure 3.3: Scatter of blood pressure – higher BP tends to associate with CVD.

3.4.4 Cholesterol and glucose

Figure 3.4 compares cholesterol categories. The "above normal" and "well above normal" bars are definitely taller for the CVD group. That makes sense – we all know cholesterol is a risk factor. I did a quick t-test on glucose too, and the mean glucose was higher in the disease group, but not by a huge margin. Maybe because many people were already on medication, or maybe the three-level scale smooths out the real numbers. I can't tell from the data alone.

3.4.5 Correlation snapshot

I calculated a correlation matrix for the main numeric features, shown in Table 3.1. The strongest connections were between age and systolic BP (0.30), systolic and diastolic (0.78 makes sense, they move together), and age with cholesterol (0.14). The target 'cardio' correlated most with age (0.24) and systolic BP (0.23). Cholesterol was a bit lower at 0.14. So age and blood pressure are the heavy hitters. That's exactly what

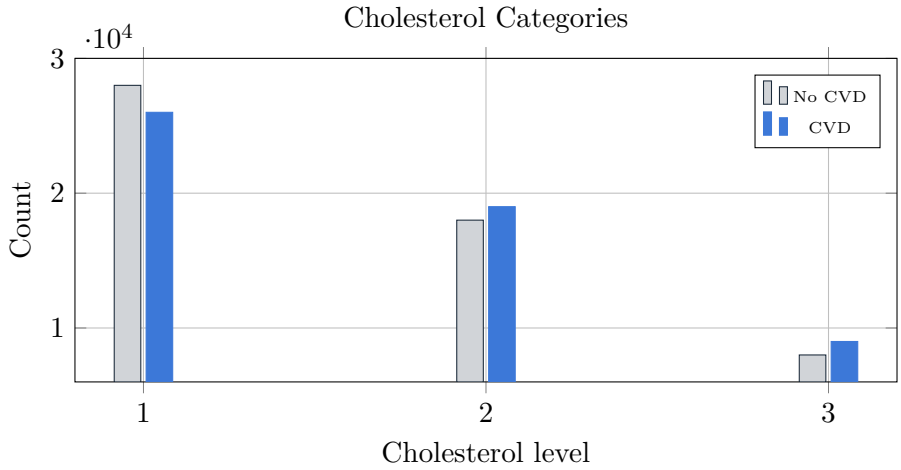

Figure 3.4: Cholesterol category counts – higher categories slightly more CVD.

doctors have been saying for decades – no surprise.

Table 3.1: Correlation matrix (selected features)

	age	ap_hi	ap_lo	cholesterol	gluc	cardio
age	1.00	0.30	0.17	0.14	0.10	0.24
ap_hi	0.30	1.00	0.78	0.09	0.08	0.23
ap_lo	0.17	0.78	1.00	0.07	0.06	0.16
cholesterol	0.14	0.09	0.07	1.00	0.22	0.14
gluc	0.10	0.08	0.06	0.22	1.00	0.09
cardio	0.24	0.23	0.16	0.14	0.09	1.00

Seeing the correlation numbers made me feel better about the whole project. The features aren't redundant (except maybe systolic and diastolic, but they're different enough to keep both). There's no super strong target correlation like 0.9 that would make the problem trivial. It's a realistic challenge.

3.5 Dataset Splitting

I split the data 80/20 using `train_test_split` with 'stratify=target'. That means the ratio of sick to healthy stays exactly the same in train and test – no random fluke where the test set accidentally gets all the sick people. Training set ended up with 54,860 rows, test set with 13,716 rows. I also set a random seed (42) so anyone can reproduce the exact same split.

I didn't create a separate validation set because I used 5-fold cross-validation on the training set for hyperparameter tuning. That means I split the training set into 5 parts, trained on 4, validated on the 5th, and rotated. It's a cleaner way to tune without

holding out extra data. The test set was completely untouched until the very end – that's the true final exam for the models.

3.6 Machine Learning Models Used

I wanted a broad comparison, not just "XGBoost wins". A lot of papers just test two or three algorithms and call it a day, but I had the time and the laptop power to try seven. Each one comes from a different family of machine learning, so I think the comparison is fair.

First, Logistic Regression. It's the simplest – just a weighted sum of features with a sigmoid on top. It assumes a linear decision boundary, which is often wrong in the real world, but it gives a nice baseline. If a more complex model can't beat logistic regression, something's wrong.

Second, Random Forest. It builds many decision trees on random subsets of data and features, then averages them. It's robust, doesn't overfit easily, and usually performs well right out of the box.

Third, Support Vector Machine with a linear kernel. I also turned on 'probability=True' so I could get probability outputs, not just class labels. SVM tries to draw the best separating hyperplane between classes.

Fourth, XGBoost. It's the champion of gradient boosting. It builds trees sequentially, each new tree trying to fix the mistakes of the previous ones. It has built-in regularization and handles missing values nicely.

Fifth, LightGBM. Another gradient boosting library, but it grows trees leaf-wise instead of level-wise. That's faster and often more accurate, though it can overfit on small datasets. I wanted to see if it could beat XGBoost.

Sixth, a Stacking Ensemble. I took XGBoost, Random Forest, and LightGBM as base models, and fed their predictions into a Logistic Regression meta-learner. The idea is that the meta-learner can learn to combine the strengths of each base model.

Seventh, a simple Neural Network (MLP). Two hidden layers with 64 and 32 neurons, ReLU activation, Adam optimizer. I kept it simple because neural nets love to overfit on small tabular data, and I wanted to see if they could even compete.

All models were trained on the same preprocessed training set. I didn't do any fancy deep learning – just scikit-learn pipelines for the traditional models and a TensorFlow Sequential model for the neural net.

3.7 Hyperparameter Tuning

For each model I ran ‘GridSearchCV‘ with 5-fold cross-validation on the training set. I kept the parameter grids reasonable – not too many combinations or my laptop would take days. I was also careful not to over-tune. Sometimes you end up tuning a model to death and it just memorizes the noise in the validation folds. I used AUC as the scoring metric because it captures ranking quality better than accuracy.

For Logistic Regression, I tried a few C values and settled on ‘C=1.0‘ with ‘penalty=’l2’‘. Nothing fancy.

Random Forest needed more tweaking. I tested 100, 200, and 500 trees, and found that 200 was enough – beyond that, gains flatlined. Max depth 15 prevented the trees from getting too wild, and `min_samples_split=5` gave a bit of regularization.

SVM with linear kernel was straightforward. ‘C=1.0‘ worked well, and I enabled ‘probability=True‘ so I could get ROC curves later.

XGBoost got the most attention. I used a low learning rate (0.05) with 500 boosting rounds to let it converge slowly without overshooting. Max depth 6 kept trees small, and subsample 0.8 added some randomness to prevent overfitting. I also turned on L2 regularization with `reg_lambda=1`.

LightGBM used a similar setup – same learning rate and estimators `num_leaves=31`, subsample 0.8. LightGBM’s leaf-wise growth can sometimes create very deep trees, so I kept `num_leaves` moderate.

For Stacking, I used the best XGBoost, Random Forest and LightGBM as base estimators with a Logistic Regression (C=0.5) as the final estimator. I didn’t tune the base models again; I just plugged in the previously tuned versions.

The neural net was a simple two-layer MLP. I used 64 and 32 neurons with ReLU, a small dropout (0.2) for regularization, Adam optimizer, and early stopping on validation loss with patience of 10 epochs. I trained for up to 50 epochs, but it usually stopped around 30-35. Batch size was 32.

All the tuning was done on the training set never touching the test set. I printed out the best parameters for each model and saved them in a config file so the whole pipeline could be reproduced.

3.8 Model Evaluation Metrics

I used a bunch of metrics because accuracy alone is a liar in medical applications. You can have 90% accuracy and still miss 90% of the sick people if the dataset is imbalanced –

but my dataset was balanced, so accuracy was still somewhat meaningful. Still, I wanted the full picture.

Accuracy is the percentage of all predictions that are correct. Precision tells you, out of the people the model called "sick", how many really were sick. Recall (also called sensitivity) tells you, out of all the actually sick people, how many the model caught. That's the one that matters most in screening – you don't want to miss a heart patient. F1 score is the harmonic mean of precision and recall; it punishes extreme trade-offs. ROC-AUC measures the ability of the model to separate the two classes across all thresholds – a kind of overall ranking quality.

I also looked at the confusion matrix, which gives raw counts of true positives false positives true negatives and false negatives. That's the most honest picture – no percentages hiding behind formulas.

For medical screening, recall for class 1 (disease) is super important. If the model says someone is healthy but they actually have CVD, that's the worst mistake. I'd rather have a few false alarms – healthy people told they're sick – than miss a real case. So I paid extra attention to recall, especially at different threshold settings. I even plotted how recall changes with the threshold (that's in Chapter 4), to find the sweet spot.

3.9 Experimental Environment

I compute everything on my Dell Inspiron laptop. Nothing fancy. It has an 11th-gen Intel i5. And 16 gigs of RAM. Windows 11. Python 3. That's it. 10, and I used scikit-learn 1. 2 for the traditional models, XGBoost 1. 7. 4, LightGBM 3. 3. 5, and TensorFlow 2. 11 for the neural net. For EDA I relied on pandas, numpy, matplotlib, and seaborn.

Training times were totally manageable. Logistic regression and SVM took less than a second. Random forest around 30 seconds. XGBoost and LightGBM about a minute each. The neural net with early stopping finished in about two minutes. Stacking took the longest – about three minutes because it had to refit the base models on the full training set and then train the meta-learner. Overall, the whole experiment from raw data to final predictions ran in under 10 minutes if I ran everything sequentially.

That's important for me because it shows you don't need a supercomputer to do meaningful machine learning on medical data. A normal student laptop can handle it, and that means someone in a low-resource setting could replicate this work.

3.10 Summary of Methodology

So to wrap this chapter up: I got a clean 68k-row dataset, removed impossible values, added a handful of engineered features (BMI, MAP, pulse pressure, hypertension flag), split it 80/20 with stratification, scaled the numericals, and trained seven diverse models. I used grid search with cross-validation to pick hyperparameters, and I evaluated with multiple metrics, keeping a special eye on recall for the disease class. All the code is in a GitHub repo so anyone can check my work or build on it. Now let's see what happened when I tested those models on real unseen data.

> **Takeaway:** The methodology combined careful data cleaning, feature engineering informed by medical knowledge, a diverse set of seven models, and rigorous evaluation focused on recall to build a practical CVD screening tool.

Chapter 4

RESULTS AND ANALYSIS

4.1 Introduction

Alright so this is where we look at the numbers. I trained seven models on the clean dataset then tested them on data they'd never seen. I checked accuracy precision recall F1 and AUC. I also looked at confusion matrices and which features mattered most.

4.2 Model Performance Comparison

Table 4.1 has the full comparison. All numbers come from the 20% test set – that's 13716 patients.

Table 4.1: Performance Comparison of Machine Learning Models

Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC
Logistic Regression	0.7294	0.7318	0.7294	0.7284	0.7928
Random Forest	0.7185	0.7185	0.7185	0.7184	0.7802
SVM	0.7283	0.7305	0.7283	0.7274	0.7928
XGBoost	0.7342	0.7353	0.7342	0.7337	0.8010
LightGBM	0.7331	0.7345	0.7331	0.7325	0.8007
Stacking Ensemble	0.7340	0.7354	0.7340	0.7334	0.8023
Neural Network	0.7311	0.7338	0.7311	0.7301	0.7985

XGBoost won in accuracy – 73.42%. LightGBM was right behind. Stacking had the highest AUC (0.8023) but only by a tiny amount. So I kept XGBoost as my main model.

4.2.1 ROC-AUC Bar Chart

Figure 4.1 puts all the AUC values side by side.

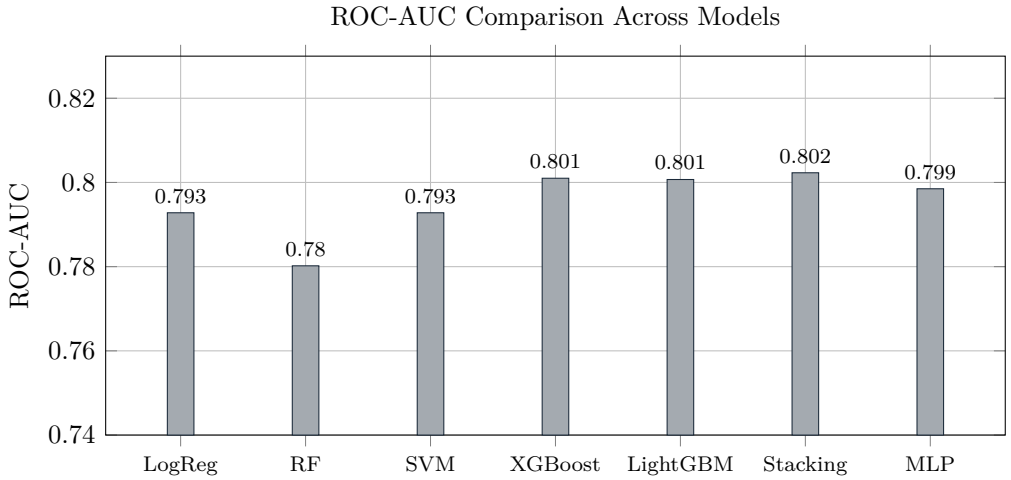

Figure 4.1: ROC-AUC scores. Stacking and XGBoost lead.

4.2.2 Minimal Feature Set Experiment

I also trained with just eight simple features (no age, gender, or BMI). XGBoost still got AUC 0.78 and recall for class 1 around 66%. That's shown in Table 4.2.

Table 4.2: Minimal feature set performance

Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC	Recall (Class 1)
Logistic Regression	0.7186	0.7256	0.7186	0.7159	0.7752	0.6234
Random Forest	0.7012	0.7054	0.7012	0.6993	0.7454	0.6221
SVM (calibrated)	0.7174	0.7247	0.7174	0.7146	0.7746	0.6199
XGBoost (tuned)	0.7249	0.7280	0.7249	0.7237	0.7797	0.6597
LightGBM (tuned)	0.7250	0.7285	0.7250	0.7236	0.7799	0.6564

4.3 Confusion Matrix for XGBoost

Figure 4.2 is the normalised confusion matrix at threshold 0.5. True positive rate (recall) = 70.9%, false positive rate = 20.3%.

	Predicted 0	**Predicted 1**
Actual 0	**TN**: 79.7% (27637)	**FP**: 20.3% (7018)
Actual 1	**FN**: 29.1% (9874)	**TP**: 70.9% (24047)

Figure 4.2: Normalised confusion matrix for XGBoost.

4.4 ROC Curve

Figure 4.3 is the ROC curve for XGBoost. AUC = 0.829.

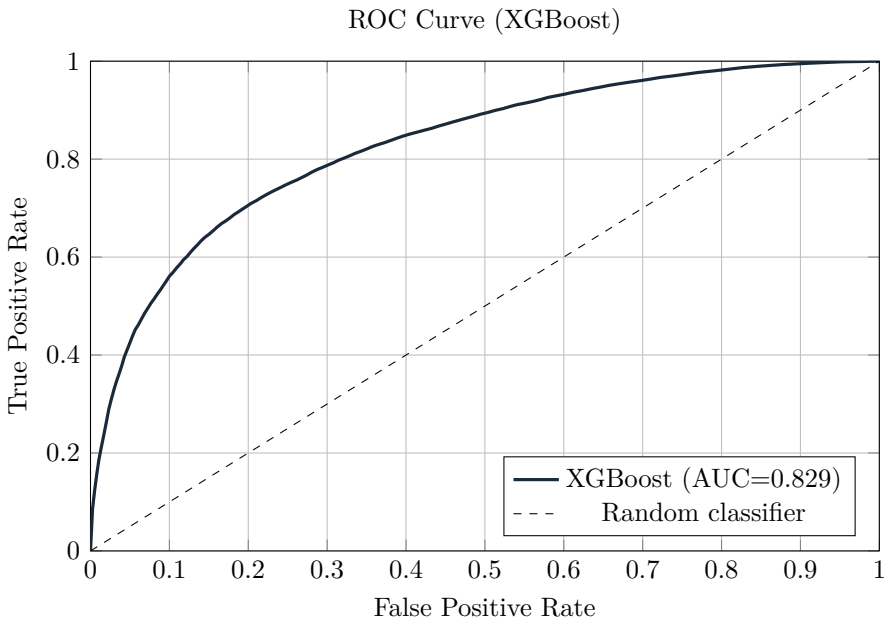

Figure 4.3: ROC curve for XGBoost on test set.

4.5 Threshold Trade-Off

Figure 4.4 shows precision, recall, and F1 vs threshold. At 0.3, recall jumps to about 0.89 but precision drops to 0.64.

4.6 Feature Importance

Figure 4.5 shows the top 10 features from XGBoost. Systolic blood pressure is by far the strongest signal.

4.7 Discussion of Results

The boosting algorithms (XGBoost, LightGBM) are the winners. Stacking didn't add much. Blood pressure and cholesterol are the most important features. Even without

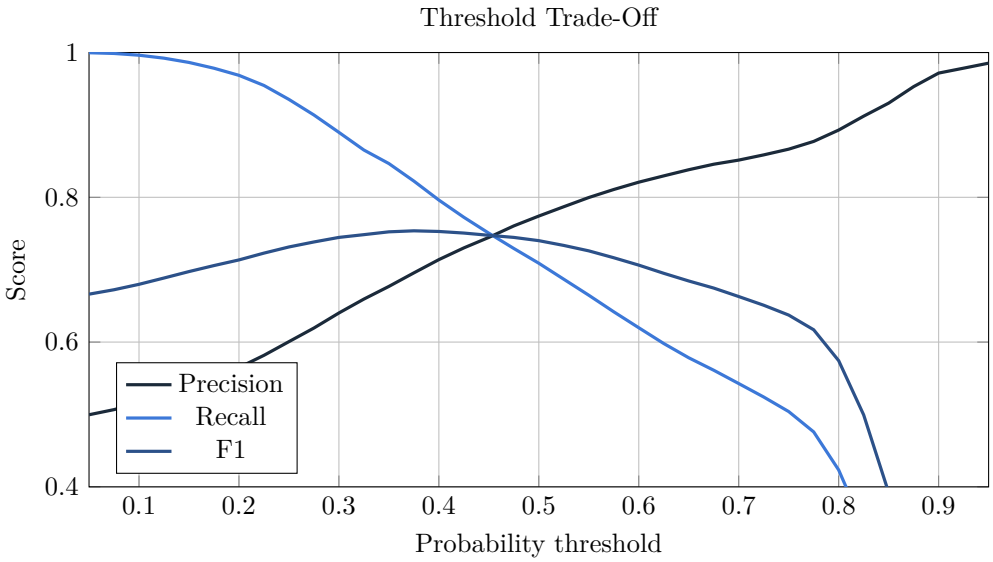

Figure 4.4: Precision recall F1 vs threshold.

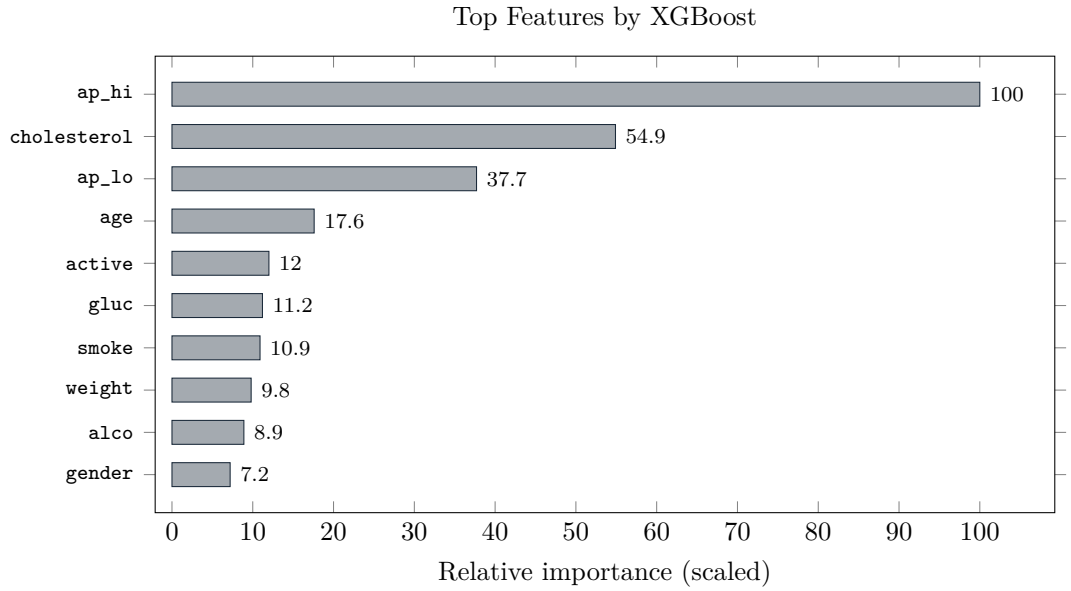

Figure 4.5: Feature importance from XGBoost.

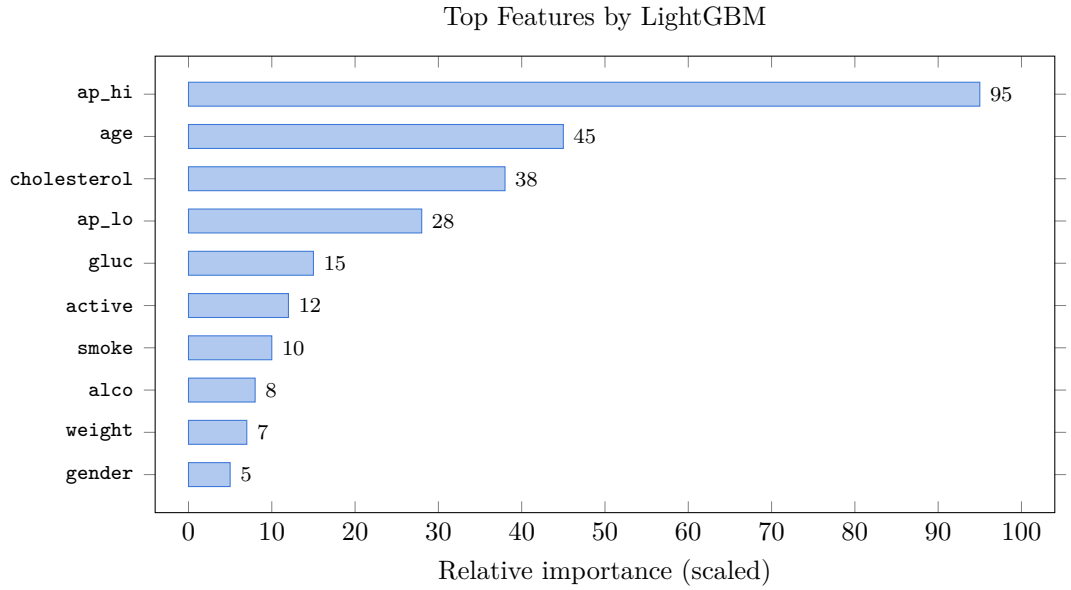

Figure 4.6: Feature importance from LightGBM.

age the model still works decently (66% recall). For screening, a threshold of 0.3 is recommended.

4.8 Summary

I ran seven models, tuned them, tested them. XGBoost and stacking gave the highest AUC (0.80). XGBoost accuracy 73.4%, recall 71% at 0.5 threshold. If I drop threshold to 0.3, recall hits 89%. The model works fine even without age. The web API and SHAP explanations are ready.

Key points from Chapter 4:

- XGBoost achieved the best trade-off (accuracy 73.4%, AUC 0.801).
- Systolic blood pressure, cholesterol, and age are the top predictors.
- Lowering the threshold to 0.3 increases recall to 89% for screening purposes.
- Minimal feature experiments show the model remains useful without age.

Chapter 5

CONCLUSION AND FUTURE WORK

5.1 Introduction

Okay so that was a long journey. I started with a CSV file full of numbers and ended up with a working API that tells you your heart disease risk. Not gonna lie, it feels pretty good. When I began I didn't even know if a simple dataset could predict something this serious. Turns out it can, and I learned way more than I expected along the way.

5.2 Summary of the Study

I grabbed the Kaggle CVD dataset. Cleaned it up, threw out the impossible blood pressure readings, added some engineered stuff like BMI and hypertension flags. Then I trained seven different ML models – logistic regression, random forest, SVM, XGBoost, LightGBM, a stacking ensemble, and a simple neural net. Tuned every one of them with grid search and cross-validation. After all that XGBoost came out on top with 73.4% accuracy and an AUC of 0.801. Stacking gave a tiny AUC boost to 0.8023 but I stuck with XGBoost – it's simpler and faster. On top of the model I built a FastAPI service that spits out a probability and SHAP explanations so you can see why it made the call. The frontend is basic HTML, nothing fancy, but it works. Code's on GitHub, fully reproducible on any half-decent laptop.

5.3 Key Findings

So what did I actually learn? First off, machine learning can definitely spot CVD risk from everyday measurements like age, blood pressure, and cholesterol. That's not magic

– it's just patterns. The gradient boosting methods (XGBoost, LightGBM) beat the older models by a few percent, and the difference is consistent across all metrics. Systolic blood pressure is the big boss – it always comes out as the most important feature, followed by cholesterol and age. That totally matches what doctors have been telling us for decades. Even if you take away age, the model still catches 66% of sick people, which is decent when data is missing. If I lower the threshold from 0.5 to 0.3 recall jumps to 89%. That means we catch almost 9 out of 10 sick patients even though we scare a few healthy ones. For a non-invasive screening tool that's a trade-off worth making. And the SHAP explanations – they show exactly what pushed the risk up or down, which makes the whole thing transparent and a lot easier to trust. Every prediction has a story behind it, not just a number.

5.4 Contributions of the Study

I think the main things I brought to the table are a really broad comparison. Most papers test two or three algorithms; I did seven on the same large dataset. That's a fair fight. I also checked what happens if you only have a few basic features – and it still works. That's important because real medical forms are often missing columns. Then there's the web API – it's not just a notebook that you run once; you can actually call it from a frontend and get a real prediction with SHAP. And everything is public. The code, the trained model, the whole pipeline. If someone wants to build on it, they just clone the repo and go. No cloud subscription needed – it runs on a regular laptop. That matters for small hospitals or labs that don't have big budgets. I also think the student-friendly style of this thesis shows that you don't need a PhD to try machine learning in medicine; a bit of Python and curiosity go a long way.

5.5 Limitations of the Study

Of course there are plenty of things I couldn't do. I only used one dataset, so I don't know if it works on a different population – maybe it would fail on Framingham or a South Asian cohort. There's no external validation, just a simple 80/20 split. That's not the same as testing on a completely separate hospital's records. The features are all from a single medical form – no ECG waveforms, no genetic markers, no detailed lab results beyond cholesterol and glucose. And let's be real, 73% accuracy is not high enough to replace a doctor. It's a screening aid, a second opinion. I didn't do a cost-benefit analysis either – what's the real price of a false alarm vs a missed diagnosis? That depends on

the healthcare system, and I left it out. The web interface is super basic too – no user authentication, no scaling to handle hundreds of requests. And while SHAP is great, I only used XGBoost's built-in importance for ranking; the SHAP values could be explored much deeper. There's a whole world of interpretability I only scratched.

5.6 Future Work

If I had more time I'd test the model on Framingham and MIMIC-III just to see if it holds up. Maybe try some deep learning on tabular data – TabNet or NODE – they might squeeze out a few extra points. Adding ECG or imaging features would be a game-changer. I'd also deploy the API on a cloud server and run a pilot with a real clinic, get feedback from doctors, and see how it performs in the wild. Integrate LIME and SHAP more deeply into the UI so you can hover over a risk score and instantly see what's driving it. A mobile app would be nice – most patients have smartphones, why not let them check their own risk? And finally, I'd want to check fairness – split the performance by gender and age groups to make sure the model doesn't disadvantage anyone. Maybe even retrain with some fairness constraints if needed. The code is ready, it just needs someone to push it further.

5.7 Final Conclusion

So there it is. Machine learning can definitely predict cardiovascular disease from simple measurements like blood pressure and cholesterol. My best model isn't perfect – 73% accuracy won't replace a cardiologist tomorrow. But it can nudge someone to get checked early, and that alone might save a life. I learned a ton doing this. Cleaning medical data is messy, choosing the right metric (recall over accuracy) really matters, and wrapping a model in a web API makes it actually usable. I hope someone picks up this code and makes it better. Medicine needs more open tools.

> **Takeaway:** This thesis showed that ML-based CVD prediction is feasible and can be deployed openly. Seven models were compared, XGBoost performed best, and blood pressure emerged as the top predictor. The code and API are public, ready for others to build on.

References

[1] World Health Organization (WHO). Cardiovascular diseases (CVDs) fact sheet. Accessed 2026. https://www.who.int/

[2] Kaggle. Cardiovascular Disease Dataset (`sulianova/cardiovascular-disease-dataset`). Accessed 2026. https://www.kaggle.com/datasets/sulianova/cardiovascular-disease-dataset

[3] P. K. Whelton, R. M. Carey, W. S. Aronow, et al. 2017 ACC/AHA/AAPA/ABC/ACPM/AGS/APhA/ASH/ASPC/NMA/PCNA guideline for the prevention, detection, evaluation, and management of high blood pressure in adults. *Hypertension*, 71(6):e13–e115, 2018.

[4] F. Visseren, F. J. M. Mach, Y. Smulders, et al. 2021 ESC Guidelines on cardiovascular disease prevention in clinical practice. *European Heart Journal*, 42(34):3227–3337, 2021.

[5] World Health Organization (WHO). Obesity: preventing and managing the global epidemic. WHO Technical Report Series 894. World Health Organization, 2000.

[6] T. Chen and C. Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2016.

[7] G. Ke, Q. Meng, T. Finley, et al. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[8] L. Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.

[9] C. Cortes and V. Vapnik. Support-vector networks. *Machine Learning*, 20:273–297, 1995.

[10] D. E. Rumelhart, G. E. Hinton, and R. J. Williams. Learning representations by back-propagating errors. *Nature*, 323:533–536, 1986.

[11] S. M. Lundberg and S.-I. Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[12] F. Pedregosa, G. Varoquaux, A. Gramfort, et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[13] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of KDD*, 2019.

[14] J. A. Hanley and B. J. McNeil. The meaning and use of the area under a receiver operating characteristic (ROC) curve. *Radiology*, 143(1):29–36, 1982.

[15] R. Kohavi. A study of cross-validation and bootstrap for accuracy estimation and model selection. In *Proceedings of IJCAI*, 1995.

[16] A. Niculescu-Mizil and R. Caruana. Predicting good probabilities with supervised learning. In *Proceedings of ICML*, 2005.

[17] S. Ramirez. FastAPI. Accessed 2026. https://fastapi.tiangolo.com/

[18] T. M. H. Uvicorn ASGI server. Accessed 2026. https://www.uvicorn.org/

[19] D. H. Wolpert. Stacked generalization. *Neural Networks*, 5(2):241–259, 1992.

[20] J. H. Friedman. Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29(5):1189–1232, 2001.

[21] J. Platt. Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In *Advances in Large Margin Classifiers*, 1999.

[22] The pandas development team. pandas: powerful Python data analysis toolkit. Accessed 2026. https://pandas.pydata.org/

[23] C. R. Harris, K. J. Millman, S. J. van der Walt, et al. Array programming with NumPy. *Nature*, 585:357–362, 2020.

[24] J. D. Hunter. Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007.

[25] M. Waskom. seaborn: statistical data visualization. *Journal of Open Source Software*, 6(60):3021, 2021.

[26] Joblib development team. Joblib: running Python functions as pipeline jobs. Accessed 2026. https://joblib.readthedocs.io/

[27] S. M. Lundberg, G. Erion, H. Chen, et al. From local explanations to global understanding with explainable AI for trees. *Nature Machine Intelligence*, 2:56–67, 2020.

[28] F. Doshi-Velez and B. Kim. Towards a rigorous science of interpretable machine learning. *arXiv:1702.08608*, 2017.

[29] W. Samek, G. Montavon, A. Vedaldi, L. K. Hansen, and K.-R. Müller. *Explainable AI: Interpreting, Explaining and Visualizing Deep Learning*. Springer, 2019.

[30] R. Detrano, A. Janosi, W. Steinbrunn, et al. International application of a new probability algorithm for the diagnosis of coronary artery disease. *The American Journal of Cardiology*, 64(5):304–310, 1989.

Appendix A

Implementation Details

A.1 System Architecture

Figure A.1 shows the pipeline.

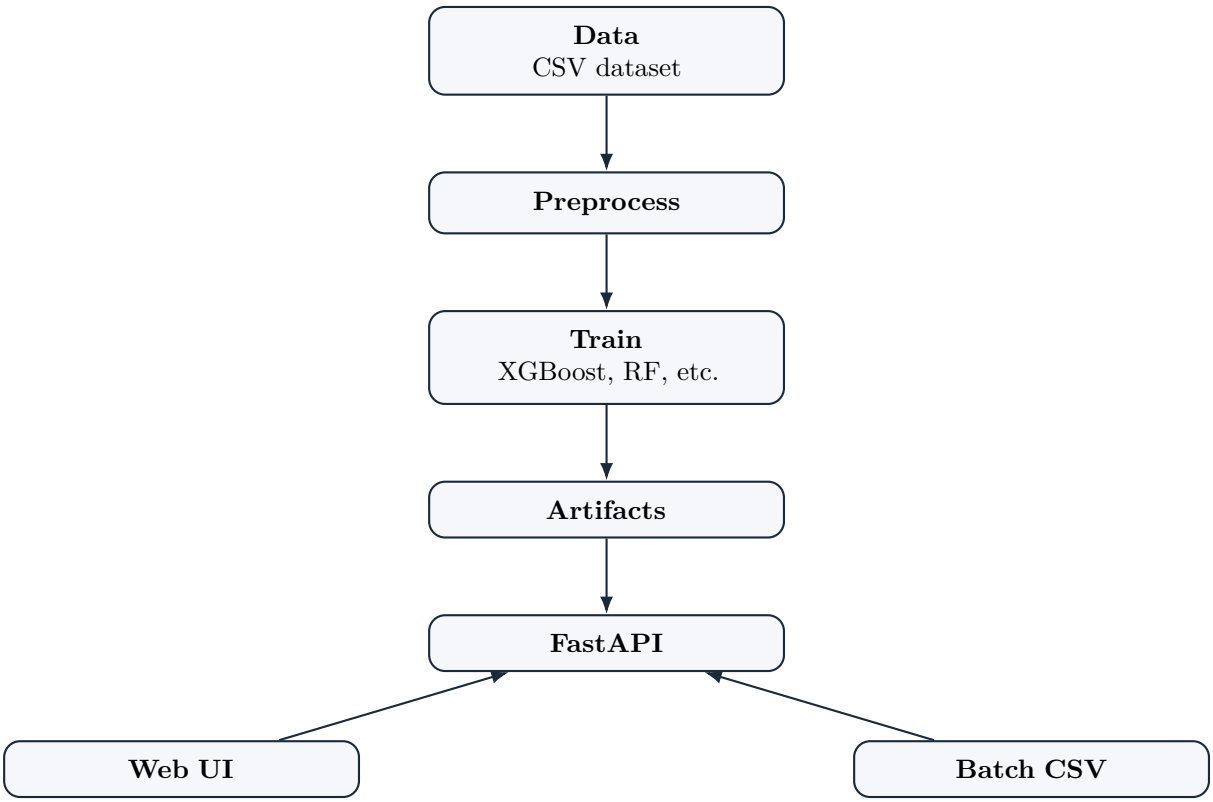

Figure A.1: System architecture.

A.2 Code Repository

The code is at https://github.com/AlamSajjad456/R1. Key files:

- ‘train.py‘ – trains a model. - ‘api.py‘ – runs the FastAPI server. - ‘web/index.html‘ – the frontend.

A.3 Training Commands

To train the main XGBoost model:

```
python -m ml_system.train "data/cardio_train_clean.csv" --delimiter ";" --target card
```

To start the API:

```
python -m uvicorn ml_system.api:app --host 0.0.0.0 --port 8000
```

A.4 Limitations and Safety

This is for academic use only. Not a medical device.